

EO-EPTC: End-to-End Original Traffic-Based Encrypted Proxy Traffic Classification Framework

Yige Chen¹, Huajie Jia, Zhenzhou Tang, *Senior Member, IEEE*, Yipeng Wang², *Senior Member, IEEE*, and Hui Liu

Abstract—Machine learning-based methods for encrypted traffic classification can be effectively applied to analyze encrypted proxy traffic generated by proxy protocols, which are intermediary protocols used to route network traffic through a remote server. Nonetheless, different encrypted proxy protocols generate distinct traffic patterns, even when they handle the same network behavior. To address these distribution differences, a straightforward approach is to collect datasets specific to each proxy protocol. However, typical proxy protocols repackage original traffic by encrypting it without payload padding or compression. This leads to a definite characteristic correlation between original and encrypted proxy traffic. We propose an End-to-end Original traffic-based Encrypted Proxy Traffic Classification framework (EO-EPTC) to bridge the distribution gap between original traffic and proxied traffic, enabling the classification of encrypted proxy traffic using a original traffic dataset. EO-EPTC conducts sequence feature alignment to reduce distribution bias and employs a Seq2Seq model to capture the underlying semantics of the proxy protocol, creating a sequence feature transformation model. We apply EO-EPTC to existing encrypted traffic classification models, training them on original traffic to classify proxied traffic. This achieves up to 99.70% accuracy on encrypted proxy traffic, comparable to models trained directly on proxied traffic.

Index Terms—Encrypted proxy, traffic classification, distribution gap, dataset generation.

I. INTRODUCTION

A. Motivation and Problem Statement

PROXY-BASED communication with encryption is a technique used to route network traffic through intermediary servers to bypass firewalls, access restricted servers and enhance anonymity. As shown in Figure 1, the firewall blocks

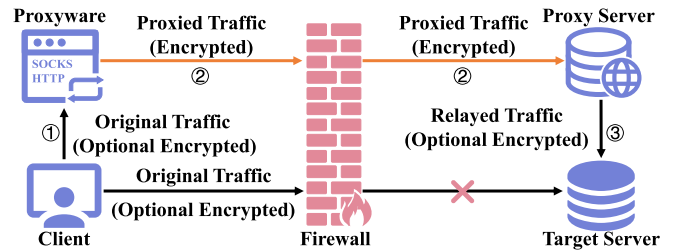


Fig. 1. Proxy-based communication with encryption.

TABLE I
DEGRADATION IN CLASSIFICATION DUE TO DISTRIBUTION GAPS

Classifier	SS	VMess	Trojan	VLESS	Classifier	SS	VMess	Trojan	VLESS
FS-Net	4.02%	4.93%	4.82%	4.08%	RF	2.96%	1.54%	1.46%	1.11%
ETC-PS	1.48%	2.22%	1.66%	2.45%	XGBoost	4.25%	3.00%	3.99%	2.87%
Transformer	4.45%	4.23%	4.07%	4.72%					

The detailed results can be found in Section VI-G.

traffic to the target server, restricting users from accessing it. To resolve this issue, users can redirect original traffic to a local proxyware using the SOCKS/HTTP protocol. The proxyware encapsulates and encrypts the traffic according to the proxy protocol and forwards it to a proxy server that is not in the firewall's jurisdiction. The proxy server decapsulates and decrypts the proxy traffic and relays the original traffic to the target server, leaving any application-layer encryption (such as HTTPS) intact between the client and the target server.

In recent years, several proxy protocols have gained popularity for their ability to circumvent network restrictions and protect user data, such as Shadowsocks [1], VMess [2], Trojan [3], and VLESS [4]. While mature technologies such as TOR [5] and VPN [6] have been extensively studied, their design goals differ from those proxy protocols examined here. TOR uses multi-hop onion routing for anonymity, whereas VPNs such as IPsec and WireGuard provide layer-3 encrypted tunnels. This study focuses on protocols that prioritize stealthiness, low latency, and adaptability to specific network conditions.

Encrypted traffic classification aims to identify the websites or applications associated with the traffic without the need for decryption [7], [8], [9]. Current machine learning-based methods have achieved high accuracy in this task. The effectiveness

Received 5 December 2024; revised 11 September 2025 and 25 November 2025; accepted 16 December 2025. Date of publication 22 December 2025; date of current version 2 January 2026. This work was supported in part by Zhejiang Provincial Natural Science Foundation of China under Grant LQN25F020027, in part by the Fundamental Scientific Research Project of Wenzhou City under Grant G2023005, and in part by the National Natural Science Foundation of China under Grant 62472011. The associate editor coordinating the review of this article and approving it for publication was Dr. Prosanta Gope. (*Corresponding author: Huajie Jia.*)

Yige Chen, Huajie Jia and Zhenzhou Tang are with the College of Computer Science and Artificial Intelligence and Wenzhou Key Laboratory for Intelligent Networking, Wenzhou University, Wenzhou 325000, China (e-mail: ygchen@wzu.edu.cn; hjia@stu.wzu.edu.cn; tzz@wzu.edu.cn).

Yipeng Wang is with the College of Computer Science and the Engineering Research Center of Intelligent Perception and Autonomous Control, Ministry of Education, Beijing University of Technology, Beijing 100124, China (e-mail: yipeng.wang1@gmail.com).

Hui Liu is with the Platform and Content Group, Tencent, Beijing 100193, China (e-mail: pvopliu@tencent.com).

Digital Object Identifier 10.1109/TIFS.2025.3646874

of the existing models heavily depends on the assumption that the training and test traffic datasets share the same distribution. Since encrypted proxy traffic is generated by encapsulating original traffic according to proxy protocol specifications, there is a distribution gap between the two types of traffic. Table I illustrates the degradation in classification when classifiers trained on original traffic are applied to proxied traffic in our collected dataset. An intuitive approach to addressing this issue is to collect proxied traffic samples for each encrypted proxy protocol separately. However, this limits data collection to an active mode and increases the cost of collection linearly with the number of protocols and classification targets.

B. Our Insights and Proposed Approach

1) *Key Observation to Encrypted Proxy Traffic:* The encapsulation mechanism of proxy protocols mainly causes the distribution gap between original traffic and proxied traffic. To classify traffic for a specific encrypted proxy protocol, it is necessary to train a model on a dataset consisting of that encrypted proxy protocol's traffic. It is a key observed fact that encrypted proxy protocols typically do not perform payload padding or compression to improve bandwidth utilization and simplify the protocol structure. As a result, there exists a correlation between the sequence features of the two types of traffic. This fact makes it feasible to construct a model for generating proxy traffic sequence features based on a dataset containing both original and proxied flows.

2) *Key Insight Into Our Approach:* A key insight into our approach is that the proxy traffic is derived from the original traffic, and the transformation between their sequence features is primarily driven by the transmission protocol and the proxy protocol. While the transformations introduced by the transmission protocol can be addressed by parsing the traffic according to the protocol specifications, the changes caused by the proxy protocol, which may include both linear and nonlinear alterations, require a more sophisticated solution. To address this, we can leverage a Seq2Seq model to capture and learn the proxy protocol's semantic transformations from the original traffic. This approach is particularly effective when sufficient traffic data is available, as it enables the model to accurately learn the complex, nonlinear relationships between the original and proxy traffic sequences.

3) *Brief Introduction to Our Approach:* In this paper, we propose the End-to-end Original Traffic-based Encrypted Proxy Traffic Classification framework (EO-EPTC), which aims to reduce the cost of dataset collection for encrypted proxy traffic classification while ensuring high classification performance. EO-EPTC leverages two sequence feature alignment algorithms and an end-to-end Seq2Seq model to learn the transformation patterns between original and proxied traffic, bridging the distribution gaps. EO-EPTC consists of four phases:

1) **Phase 1: Sequence Feature Alignment.** This phase aligns the sequence feature pairs of original traffic and proxied traffic to address the inconsistencies caused by network factors and protocol handshakes. The sequence feature alignment helps the subsequent model learn more

stable transformation patterns from sequence feature pairs.

2) **Phase 2: Sequence Feature Transformation Modeling.** This phase trains a Seq2Seq model to learn the underlying semantics of proxy protocols from the aligned sequence feature pairs. The Seq2Seq model bridges the distribution gap between original traffic and proxied traffic, allowing the generated sequence features to better simulate proxy behaviors.

3) **Phase 3: Simulated Sequence Feature Generation.** In this phase, the Seq2Seq model, trained during the transformation phase, utilizes original traffic sequence features to generate simulated proxied traffic sequence features. These simulated features closely resemble real proxied traffic and serve as a low-cost alternative for training classifiers without relying on real-world encrypted proxy traffic datasets.

4) **Phase 4: Proxied Traffic Classification.** In this phase, we use the simulated proxied traffic sequence features to train existing encrypted traffic classifiers. Based on EO-EPTC, these classifiers can handle various proxy protocols and adapt to real-world traffic patterns.

C. Key Contributions

We briefly summarize our main contributions as follows:

- We propose an End-to-end Original traffic-based Encrypted Proxy Traffic Classification framework (EO-EPTC) to bridge the distribution gap between original traffic and proxy traffic, enabling the training of an encrypted proxy traffic classification model using only a original traffic dataset.
- EO-EPTC uses sequence alignment algorithms to pre-align the sequence features of original and proxied traffic, then builds a Seq2Seq model to adaptively learn the underlying semantics of proxy protocols. EO-EPTC enables the automatic generation of simulated sequence features corresponding to proxied traffic, which significantly reduces the cost of collecting datasets.
- We conduct comprehensive evaluation experiments on EO-EPTC. The results demonstrate that existing encrypted traffic classification models can achieve up to 99.70% accuracy when trained on the original traffic dataset to classify proxied traffic.

The remaining sections of this paper are structured as follows. In Section II, we present an overview of prior works. In Section III, we present the design of EO-EPTC, offering insights into the system's architecture and providing an in-depth explanation of each phase. In Section IV, we elaborate on the details of our dataset collection and provide an overview of the datasets used in this study. Section V presents the comparative experimental settings and provides a comprehensive analysis of the evaluation results. Section VI presents the performance evaluation for EO-EPTC. Finally, Section VII outlines the limitations and future work.

II. RELATED WORK

In this section, we first provide an overview of current methods for encrypted proxy traffic classification. Subsequently, we

discuss existing approaches for encrypted traffic classification in general. Additionally, we introduce several mainstream encrypted proxy protocols.

A. Encrypted Proxy Traffic Classification

The study of encrypted proxy traffic classification has gained significant attention due to the increasing use of proxy protocols to bypass firewalls and ensure privacy. Researches [10], [11], [12], [13] have proposed several methods to identify proxied traffic.

In 2022, Zhao et al. [11] examined deep learning methods for identifying abnormal encrypted proxy traffic with high accuracy, specifically targeting Shadowsocks and VMess. In 2023, Ma et al. [12] proposed a random flow rerouting defense to enhance the flow-context-aware WFP system by significantly reducing the accuracy of attacks, while incurring minimal bandwidth and latency overhead. In 2024, Xue et al. [13] demonstrated a protocol-agnostic approach to fingerprinting obfuscated traffic, targeting Shadowsocks, VMess, Trojan, and VLESS protocols by leveraging encapsulated TLS handshakes.

Existing studies on encrypted proxy traffic classification are often designed and evaluated under protocol-specific datasets, making their effectiveness highly dependent on the availability of labeled traffic for each proxy protocol. This reliance not only increases the cost of data collection but also restricts the generalization of these methods when applied to heterogeneous proxy protocols.

B. Encrypted Traffic Classification

Encrypted traffic classification methods can be categorized based on the features they utilize, including statistical features, payload features, and sequence features.

Statistical feature-based approaches [14], [15], [16], [17], [18], [19], [20], [21], [22], [23] rely on the statistical characteristics of traffic flows to classify encrypted traffic. In 2020, Montieri et al. [14] proposed a hierarchical traffic classification framework for anonymity tools, demonstrating significant improvements over flat classification methods. Shapira and Shavitt [20], [21] proposed FlowPic, which transforms per-flow packet size and arrival time statistics into image-like representations and leverages a CNN to accurately classify encrypted Internet traffic into categories and applications, even in the presence of VPN and Tor. Babaria et al. [22] proposed FlowFormers, which first convert bidirectional flows into FlowPrints, i.e., time-length histograms of up/down packet, byte, and average-length counters over packet-size bins, and then feed these behavioural sequences into transformer-augmented CNN/LSTM for fine-grained application-type classification. In 2022, Horowicz et al. [23] proposed a few-shot traffic classification framework based on mini-FlowPic representations and network-aware augmentations, which reduces dependency on large labeled datasets while improving accuracy and implementation efficiency. In 2024, Cerasuolo et al. [18] proposed MEMENTO, a class incremental learning framework for encrypted traffic, improving new app classification and stability on old apps through bi-flow augmentation and enhanced distillation.

Payload feature-based approaches [24], [25], [26], [27], [28], [29], [30], [31], [32] enhance classification accuracy by analyzing packet content, size, and structure. In 2021, Aceto et al. [24] presented a multitask multi-modal deep learning model for encrypted traffic, improving performance on various datasets over single-modality approaches. In 2022, Xiao et al. [25] developed EBSNN, a neural network with segmented byte encoding and attention, achieving remarkable performance in application and website classification. In 2022, Lin et al. [28] introduced ET-BERT, a Transformer pre-trained on datagram-level data, achieving strong generalization and accuracy across tasks in encrypted traffic classification after fine-tuning.

Sequence feature-based approaches [33], [34], [35], [36], [37], [38] leverage characteristics such as packet length, message type, and arrival timing to model encrypted traffic, effectively capturing temporal dependencies and enhancing classification accuracy. In 2019, Liu et al. [33] introduced FS-Net, an RNN-based network capturing flow sequences, achieving high true positive rates on encrypted traffic datasets with an encoder-decoder design. In 2022, Xu et al. [35] developed ETC-PS, leveraging path signatures on packet sequences for robust encrypted traffic classification across various protocols. In 2023, Zhou et al. [38] presented WF-Transformer for Tor traffic classification, using temporal features to outperform CNNs in both closed-world and open-world scenarios.

Research on encrypted traffic classification has advanced through statistical, payload, and sequence features, and these approaches demonstrate strong performance when training and testing distributions are aligned. However, they provide limited adaptability to asymmetric scenarios, such as when only original traffic datasets are available for training while the target traffic is proxied, leaving a gap for methods that can explicitly bridge this distribution difference.

C. Encrypted Proxy Protocols

Encrypted proxy protocols ensure secure, private communication between clients and servers by protecting data from tampering and interception. Protocols like Shadowsocks [1], VMess [2], Trojan [3], and VLESS [4] are commonly used in proxyware to balance security, performance, and obfuscation.

Shadowsocks [1] is a lightweight proxy protocol that uses symmetric encryption algorithms like AES-256-GCM to secure communication. It incorporates traffic obfuscation techniques to evade detection by firewalls.

VMess [2], the core protocol of V2Ray [39], enables encrypted communication with robust encryption and authentication mechanisms. It supports dynamic key exchange alongside TLS to enhance stealth and resist traffic analysis.

Trojan [3] disguises proxy traffic as legitimate HTTPS traffic by enforcing strict use of TLS. By mimicking HTTPS behavior, it becomes difficult for Deep Packet Inspection (DPI) systems to distinguish it from regular web traffic.

VLESS [4] is a streamlined variant of VMess that eliminates authentication and dynamic encryption to reduce latency. It is commonly paired with TLS to maintain security and is often used in scenarios where low latency is prioritized.

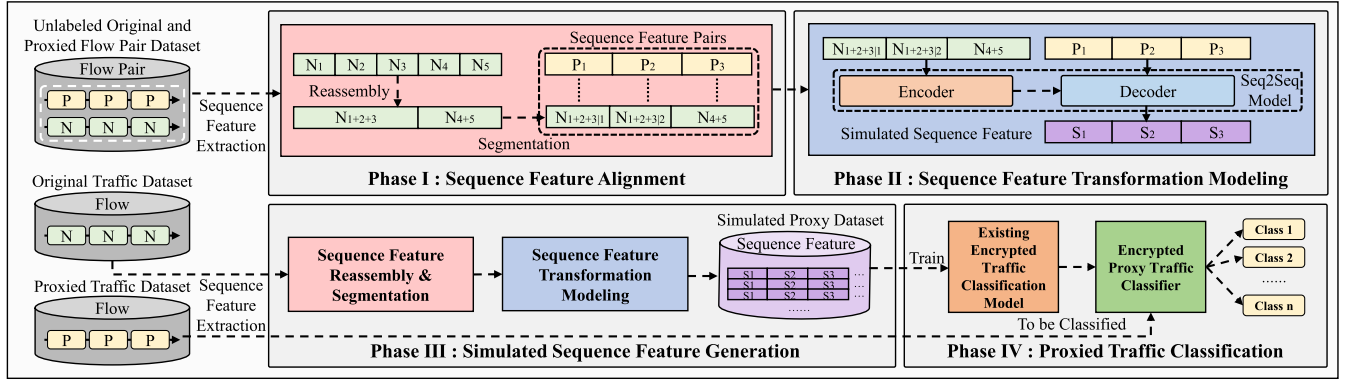


Fig. 2. Architecture of end-to-end original traffic-based encrypted proxy traffic classification framework (EO-EPTC).

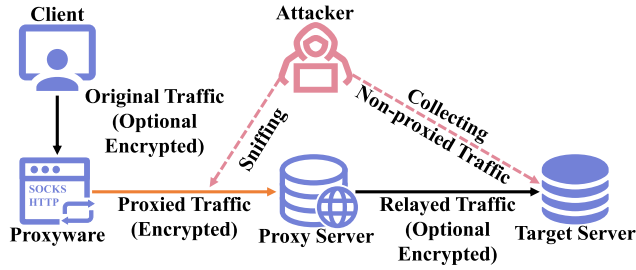


Fig. 3. The threat model of encrypted proxy traffic classification based on original traffic.

Mainstream encrypted proxy protocols repackage original traffic by encrypting it without payload padding or compression. This preserves certain correlations such as packet length patterns between original traffic and proxied traffic. These shared characteristics across different proxy protocols reveal an opportunity to leverage original traffic for building models that remain effective on encrypted proxy traffic.

III. SYSTEM FRAMEWORK

In this section, we first introduce the threat model for classifying encrypted proxy traffic using original traffic. Then, we describe the structured design of the proposed classification framework, EO-EPTC.

A. Threat Model

Figure 3 illustrates the scenario in which a client attempts to establish a secure communication channel with the target server, using both the proxyware and the proxy server as intermediaries. Proxy refers to the intermediate relay mechanism in which a client forwards original traffic to a proxy server using a specified protocol. The proxy server then decapsulates, transmits, and delivers the traffic on behalf of the client. During the proxy communication process, both the proxyware and the proxy server must adhere to a predefined proxy protocol, such as VMess [2]. The proxyware encapsulates network packets intended for the target server according to the specified proxy protocol and sends them to the proxy server, which then decapsulates and forwards them to the target server.

This paper considers an attacker capable of sniffing encrypted proxy traffic at positions in the network through

which it passes (e.g., firewalls or gateways). The attacker can observe IP header information and side-channel metadata but cannot compromise the shared secrets between communicating parties or decrypt the payload content of the encrypted proxy traffic.

Before launching the actual attack, the attacker is assumed to locally access the target applications without using a proxy and collect their original traffic. Although this original traffic may itself be encrypted (e.g., HTTPS/TLS), it remains valuable because flow-level characteristics, such as packet size distributions, inter-arrival times, and directional patterns, are observable without decryption. Using these flows, the attacker can construct a labeled dataset at relatively low cost.

With this dataset, the attacker trains a multi-class classifier to capture application-specific traffic patterns. During the attack phase, when encrypted proxy traffic is available at the observation point, the trained classifier is applied to infer which application the intercepted traffic corresponds to. Crucially, the analysis relies exclusively on flow statistics and side-channel features, rather than any payload content.

B. Overview

As shown in Figure 2, EO-EPTC consists of four main phases: Sequence Feature Alignment, Sequence Feature Transformation Modeling, Simulated Sequence Feature Generation, and Proxied Traffic Classification.

Sequence Feature Alignment: This phase employs sequence feature reassembly and segmentation algorithms to align the sequence features extracted from the unlabeled original and proxied flow pair dataset. The alignment mitigates inconsistencies in sequence features caused by network transmission or proxy protocol standards.

Sequence Feature Transformation Modeling: This phase uses a Seq2Seq model to capture the underlying semantic information by learning the mapping between original and proxied traffic sequence features of the above unlabeled flow pair dataset. The model employs an encoder-decoder architecture, encoding the sequence features of original traffic into an intermediate representation and then decoding it to produce the simulated sequence feature of proxied traffic.

Simulated Sequence Feature Generation: This phase takes a original traffic dataset as input and uses the Seq2Seq

model, trained during the sequence feature transformation phase, to generate simulated sequence features corresponding to the proxied traffic. The original and proxied traffic datasets are collected from the same website targets, but neither overlaps with the unlabeled flow pair dataset.

Proxied Traffic Classification: In this phase, we use the simulated proxied sequence features generated in the previous phase to train classifiers for encrypted proxy traffic. These classifiers demonstrate comparable classification performance to those trained directly on sequence features extracted from proxied traffic.

C. Sequence Feature Alignment

Uncertain factors such as proxy protocol handshakes, network transmission fluctuations, and MTU fragmentation can lead to misaligned or unalignable sequence features between original and proxied traffic. We first perform sequence feature extraction separately for both original and proxied traffic, followed by sequence feature reassembly and segmentation for the original traffic. The alignment of sequence features helps simplify the learning of the mapping relationship between the two types of traffic, reducing errors from incidental sequence misalignments during subsequent Seq2Seq model training and prediction.

1) *Sequence Feature Extraction:* The encrypted proxy client locally receives connections to be proxied via SOCKS or HTTP, which is the original traffic. It then encapsulates the traffic payload according to the proxy protocol specifications before forwarding the traffic to the proxy server, thereby resulting in the proxied traffic.

Therefore, for original TCP and UDP traffic, we extract the sequence of traffic payload lengths as the sequence feature. Packet directions are explicitly denoted with “+/-”, where “+” represents client-to-server (C2S) packets and “-” represents server-to-client (S2C) packets. For encrypted proxy traffic, the sequence features are derived based on the protocol type. For proxy protocols running over TCP or UDP, we extract the sequence of payload lengths from the stream, similar to the method used for original traffic. For proxy protocols operating over TLS, we use the sequence of payload lengths within the TLS stream as the sequence feature, specifically the lengths of messages carried in application data.

2) *Sequence Feature Alignment:* Sequence feature alignment includes both reassembly and segmentation processes. The reassembly refers to reconstructing fragmented or segmented payloads into complete data units based on protocol rules, ensuring the correct application-level payload boundaries. The segmentation follows reassembly to handle any overly long payloads by dividing them into smaller, consistent units to correct over-reassembly caused by protocol uncertainties.

In the TCP/IP protocol architecture, the application layer handles application-specific functions, abstracting away the details of data transmission. It hands over the payload transmission to the transport layer, which segments the payload according to the network link’s Maximum Transmission Unit (MTU). According to TCP protocol specifications, the final segment is marked with the TCP push flag, signaling the

Algorithm 1 Sequence Feature Reassembly

Require: Packet sequence $\mathbf{P} = \{p_1, p_2, \dots, p_n\}$. Each packet p_i has attributes l_i (length), d_i (direction), and f_i (push flag).
Ensure: Reassembled sequence feature $\mathbf{S}$.

```

1:  $\mathbf{S} \leftarrow []$ 
2:  $c2s\_agg \leftarrow 0, s2c\_agg \leftarrow 0$ 
3: for  $i = 1$  to  $n$  do
4:   if  $l_i > 0$  then  $\triangleright$ Dropping unloaded packets
5:     if  $d_i = c2s$  then  $\triangleright$ Distinguish packet directions
6:        $c2s\_agg \leftarrow c2s\_agg + l_i$ 
7:       if  $f_i = 1$  then  $\triangleright$ Last fragmented packet
8:          $\mathbf{S}.append(-c2s\_agg)$ 
9:          $c2s\_agg \leftarrow 0$   $\triangleright$ Reinitialize aggregation
10:      end if
11:     else if  $d_i = s2c$  then
12:        $s2c\_agg \leftarrow s2c\_agg + l_i$ 
13:       if  $f_i = 1$  then
14:          $\mathbf{S}.append(s2c\_agg)$ 
15:          $s2c\_agg \leftarrow 0$ 
16:       end if
17:     end if
18:   end if
19: end for
20: return  $\mathbf{S}$ 
```

receiver to reassemble the segmented payload and promptly deliver it to the application layer. For both original TCP traffic and TCP-based proxied traffic, we apply Algorithm 1 to reassemble sequence features, with the TCP push flag guiding the reassembly process. Here, client-to-server (C2S) packets are marked with “+” and server-to-client (S2C) packets with “-”. The reassembly of sequence features can prevent duplicate counting of the length of packet headers. For the original UDP traffic and UDP-based proxied traffic, we directly use their sequence features without performing any reassembly.

In TCP transmission, the occasional absence of the TCP Push flag can lead to over-reassembly during the above sequence feature reassembly. To address this, it is necessary to adaptively determine an appropriate threshold for reassembly payload length, as the upper limit varies with transmission configurations. In traffic datasets of statistical significance, the most frequently occurring payload length exceeding the MTU can serve as a natural upper limit. This value is selected as the reassembly payload length threshold, and any reassembled payload exceeding this limit is further segmented to mitigate over-reassembly caused by the missing Push flag. Algorithm 2 is applied to perform this segmentation, ensuring that the reassembled sequence features of original traffic align more closely with those of proxied traffic. This alignment facilitates the Seq2Seq model in effectively learning and predicting these two types of sequences.

D. Sequence Feature Transformation Modeling

After sequence feature alignment, we train a sequence feature transformation model to learn the intricate mapping rules between original and proxied sequence features. The

Algorithm 2 Sequence Feature Segmentation

Require: A set of sequence features $\mathcal{S} = \{\mathbf{S}_1, \mathbf{S}_2, \dots, \mathbf{S}_m\}$, where each $\mathbf{S}_i = \{S_{i1}, S_{i2}, \dots, S_{in}\}$ represents a sequence feature.

Ensure: The segmented sequence features $\mathcal{S}' = \{\mathbf{S}'_1, \mathbf{S}'_2, \dots, \mathbf{S}'_m\}$.

- 1: **Step 1: Calculate Global Transmission Length Threshold**
- 2: Initialize $\mathcal{L} \leftarrow \bigcup_{\mathbf{S}_i \in \mathcal{S}} \{|S_{ij}| \mid S_{ij} \in \mathbf{S}_i\}$
- 3: $\tau \leftarrow \text{mode}(\mathcal{L})$ $\triangleright$ Most frequent element in $\mathcal{L}$
- 4: $T_{\min} \leftarrow \max(\text{MTU} - \text{TCP.head})$ $\triangleright$ Minimum τ threshold
- 5: **while** $\tau \leq T_{\min}$ **do**
- 6: $\tau \leftarrow \text{next_valid_mode}(\mathcal{L}, T_{\min})$
- 7: **end while**
- 8: **Step 2: Split Sequences Based on τ**
- 9: **for** each sequence $\mathbf{S}_i \in \mathcal{S}$ **do**
- 10: Initialize an empty sequence $\mathbf{S}'_i \leftarrow \emptyset$
- 11: Initialize a temporary variable $T \leftarrow 0$
- 12: **for** each element $S_{ij} \in \mathbf{S}_i$ **do**
- 13: **if** $|S_{ij}| > \tau$ **then** $\triangleright$ Oversize according to τ
- 14: $T \leftarrow T + S_{ij}$
- 15: **while** $|T| > \tau$ **do**
- 16: **if** $T > \tau$ **then** $\triangleright$ Segment according to τ
- 17: $\mathbf{S}'_i.\text{append}(\tau)$
- 18: $T \leftarrow T - \tau$
- 19: **else if** $T < -\tau$ **then**
- 20: $\mathbf{S}'_i.\text{append}(-\tau)$
- 21: $T \leftarrow T + \tau$
- 22: **end if**
- 23: **end while**
- 24: **if** $T \neq 0$ **then**
- 25: $\mathbf{S}'_i.\text{append}(T)$
- 26: $T \leftarrow 0$
- 27: **end if**
- 28: **else**
- 29: $\mathbf{S}'_i.\text{append}(S_{ij})$
- 30: **end if**
- 31: **end for**
- 32: **end for**
- 33: **return** $\mathcal{S}'$

challenge faced by classifiers arises from the differences in traffic data distributions between original and proxied sequences, which are mainly influenced by the encapsulation mechanisms inherent to proxy protocols, rather than random variations. By extracting and aligning sequence features, we eliminate random transmission noise, improving the structural and semantic comparability between original and proxied traffic.

To address the residual differences between original and proxied sequence features, we design the Sequence Feature Transformation Model using a Seq2Seq architecture. The model learns the universal mapping relationships outlined by proxy protocol specifications, enabling the transformation of previously unseen original sequence features into simulated proxy-like features. This significantly enhances the classifiers' ability to generalize across various traffic scenarios, while

ensuring that the transformed features retain crucial classification information and address structural and semantic inconsistencies.

1) Understanding Feature Distinction: Despite sequence feature alignment addressing discrepancies caused by transmission, a distribution gap persists between original and proxied sequence features due to transformations introduced by proxy encapsulation mechanisms. Figure 4 illustrates an example of Shadowsocks sequence feature alignment, highlighting key differences:

Boundary Discrepancies: Variations at boundary positions include examples like -208 and +39 in Original (indices 0 and 1) compared to -32, -47, and -34 in Proxied (indices 0, 1, and 19). These differences arise from the handshake and teardown specifications of proxy protocol-specific initialization routines, which introduce heterogeneity in packet structures during connection establishment and termination.

Fixed-Value Bias: A uniform encapsulation mechanism introduces a fixed bias of 34 across most elements, likely resulting from the consistent addition of control fields defined by the proxy protocol or the inclusion of encryption parameters during transmission.

Variable-Value Bias: Instances like +3509 and +3575 in Original and Proxied at index 3 exhibit a variable bias of 67. This variability often stems from additional content introduced during early-stage proxy transmission. Factors like dynamic payload adjustments further amplify divergence beyond the fixed bias.

Merging and Misalignment: Examples such as double +602 in Original (indices 8 and 10) vs. +1272 in Proxied (index 9) demonstrate that proxies may reorganize segmented content during transmission. However, this reorganization inherently alters the sequence feature structure, resulting in feature order deviations and structural discrepancies.

2) Sequence Feature Transformation: We address the distribution gap between original and proxied traffic sequence features by employing an encoder-decoder Seq2Seq model with Transformer as the core sequence model.

The encoder-decoder architecture consists of two components: the encoder, which encodes original traffic sequence features into intermediate representations capturing underlying proxy semantics, and the decoder, which transforms these representations into simulated encrypted proxy traffic sequence features. This mapping can minimize the distribution gap between original and proxied traffic features without requiring prior knowledge of the target proxied traffic.

The Transformer, originally designed for natural language processing (NLP), has recently gained prominence in encrypted traffic classification, outperforming traditional RNN-based methods. While many approaches, such as ET-BERT [28] and WF-Transformer [38], utilize only the encoder combined with a classifier, EO-EPTC adopts the full Transformer encoder-decoder architecture. This enables the effective transformation of sequence features from original to proxied traffic. We employ an attention mechanism, positional encoding, and an order-agnostic architecture to address the four types of sequence feature distinctions mentioned above:

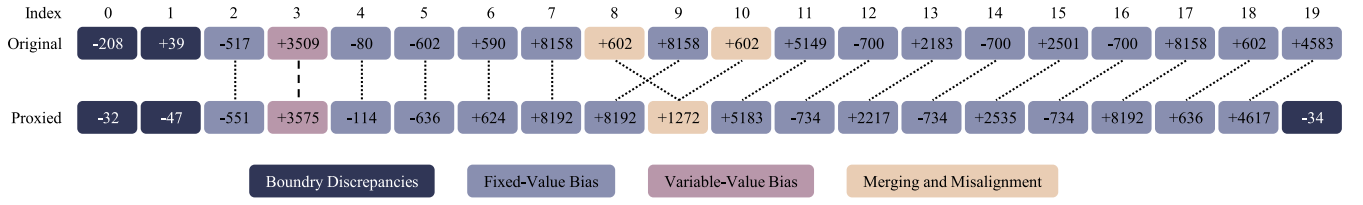


Fig. 4. An example of Shadowsocks sequence feature alignment ('+' denotes client-to-server (C2S) and '-' denotes server-to-client (S2C)).

- The context-sensitive nature of the attention mechanism dynamically captures contextual relationships between sequence features. The decoder with cross-attention extracts relevant source patterns and effectively manages dynamic deviations caused by variable-value bias. The self-attention of the Transformer can effectively handle boundary discrepancies by capturing global and local dependencies, enabling precise alignment of boundary variations caused by protocol-specific initialization. Its multi-head attention further enhances boundary modeling by focusing on sequence-specific details.
- Along with multi-head attention capturing fixed-offset patterns and other semantic features, the positional encoding of the Transformers preserves relative positional information. This enables the Seq2Seq model to adjust for the fixed-value bias introduced by proxy encapsulation.
- The order-agnostic architecture of the Transformer facilitates the identification and reconstruction of complex reordering or merging patterns within sequences. This capability ensures accurate sequence alignment, preserving structural relationships even in cases of proxy-induced misalignment or merging.

3) *Transformer Details*: The core of Transformers is the multi-head position encoding and self-attention mechanism. It further combines feedforward neural networks, residual connections, and layer normalization to learn contextual relationships. Next, this paper will elaborate on the Transformer's position encoding, self-attention, multi-head attention, and the loss function.

a) *Position Encoding*: The position of elements in a sequence holds crucial fingerprint information. Unlike RNNs, which process elements step by step, the Transformer's self-attention mechanism enables simultaneous interactions across all elements. To preserve positional and dependency information, the Transformer incorporates position encoding. For a sequence, the position encoding is calculated as follows:

$$P_{i,2j} = \sin\left(\frac{i}{10000^{2j/d}}\right), P_{i,2j+1} = \cos\left(\frac{i}{10000^{2j/d}}\right) \quad (1)$$

where i is the position of the corresponding element in a sequence, d is the word embedding length, and $j \in [1, 2, \dots, \frac{d}{2}]$.

b) *Self-Attention*: The self-attention mechanism calculates relationships across all sequence positions, representing each element based on these interactions. For each input vector X , linear transformations produce the query Q , key K , and value V vectors using weight matrices W_q , W_k , and W_v , which are initialized randomly and optimized during training.

For example, the scaled dot-product attention mechanism computes relevance using the dot product of Q and K , with the resulting scores weighting V to produce position-specific outputs, the calculation is as follows:

$$Q = W_q X, \quad K = W_k X, \quad V = W_v X$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V. \quad (2)$$

c) *Multi-Head Attention*: The multi-head attention mechanism allows simultaneous attention to multiple subspaces within the input vector. Specifically, it applies h independent linear transformations to the query Q , key K , and value V .

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$ (3)

Each head learns its own parameters, capturing dependencies at different levels or parts of the sequence. The model then concatenates the outputs of all heads and applies a linear transformation with W^O to form the final attention representation.

d) *Masked Loss*: Given the variable length of traffic feature sequences and the need to avoid introducing noise from padding elements during optimization, we adopt a masked softmax cross-entropy loss function to ensure accurate and effective model training. This approach ensures that the model is trained solely on valid sequence elements, thereby focusing the learning process on meaningful data while disregarding padding-induced artifacts. Let the original sequence feature be denoted as $x = (x_1, \dots, x_T)$, and the proxied sequence feature as $y = (y_1, \dots, y_T)$, where T represents the sequence length. The loss function is defined as:

$$\mathcal{L} = \frac{\sum_{i=1}^T w_i \cdot (-\log p(y_i | y_1, \dots, y_{i-1}, x))}{\sum_{i=1}^T w_i} \quad (4)$$

where the weight w_i masks invalid positions in the sequence. $p(y_i | y_1, \dots, y_{i-1})$ represents the conditional probability of predicting y_i , based on all previous target features $y_1, \dots, y_{i-1}$. This can be expressed as:

$$p(y_i | y_1, \dots, y_{i-1}, x) = \frac{\exp(\hat{y}_{i,y_i})}{\sum_{j=1}^V \exp(\hat{y}_{i,j})} \quad (5)$$

Here, $\hat{y}_{i,y_i}$ refers to the logit output for the target feature y_i . The denominator sums over all possible features in the feature table, indexed by j , and V is the size of the feature table.

E. Simulated Sequence Feature Generation

In this phase, we apply the Seq2Seq model, trained solely on the Unlabeled Original and Proxied Flow Pair Dataset, to

transform original sequence features into simulated proxied sequence features. The Seq2Seq model supports sequence feature transformation without relying on the prior knowledge of the target proxied traffic. The datasets of original and proxied traffic, sourced from the same website targets, can remain distinct from the unlabeled flow pair dataset.

The original traffic initially goes through sequence feature alignment, as detailed in Phase 1, to resolve inconsistencies caused by factors such as MTU fragmentation and protocol handshakes. This process also ensures compatibility between the original traffic's sequence features and the patterns learned by the Seq2Seq model.

Following sequence feature alignment, the encoder of the Seq2Seq model trained in Phase 2 converts the sequence features of original traffic into intermediate representations. The decoder then transforms these representations into simulated sequence features corresponding to the target proxied traffic. This process retains the intrinsic characteristics of original traffic while embedding proxy protocol semantics.

F. Proxied Traffic Classification

In this phase, the simulated proxied traffic sequence features generated in the previous phase are utilized to train existing encrypted traffic classification models. The simulated proxied traffic features produced by the Seq2Seq model closely mimic real proxied traffic, making them compatible with the input formats and requirements of existing methods, allowing those models to maintain their classification capabilities and advantages. Based on our experimental results, EO-EPTC exhibits strong generalization ability, achieving accurate classification performance in real-world encrypted proxy traffic scenarios while maintaining scalability and adaptability across diverse proxy protocols.

In addition, the simulated proxied traffic features can also be applied to out-of-distribution (OOD) traffic detection, enabling the identification of previously unseen traffic that does not belong to any learned class. Existing classifiers can be adapted by using their full classification probability vectors as recognition features. An additional supervised binary classifier can then be applied to identify OOD traffic. To obtain the training data for the OOD classifier, we require some samples to serve as OOD traffic so that supervised training can be performed. In practical scenarios, classes outside the target recognition set can be grouped together and treated as OOD samples. By combining the proxied traffic classifier with the OOD classifier, OOD detection can be achieved in addition to recognizing proxied traffic. Compared to threshold-based methods that rely solely on a minimum classification probability, this supervised approach makes full use of the entire probability vector, enabling more accurate and robust decisions.

IV. EXPERIMENTAL DATASETS

This section introduces how we collect an experimental dataset and provides an overview of the dataset used in the experiments.

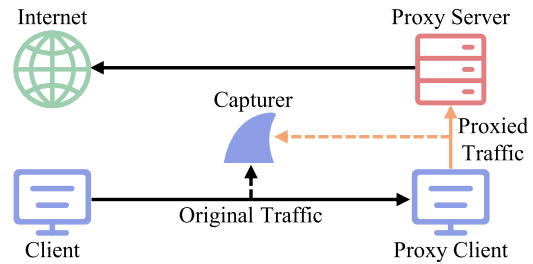


Fig. 5. The topology of dataset collection platform.

TABLE II
STATISTICAL SUMMARY OF THE TCP-BASED DATASET

Proxy Protocol	# of Webs	# of Flow Pairs	# of Original Packets	# of Proxied Packets	File Size
SS	60	1,924,994	129,188,898	143,144,593	417.43GB
VMess	60	1,447,650	116,895,522	36,831,908	466.85GB
Trojan	60	1,497,140	119,277,887	38,063,133	457.47GB
VLESS	60	1,504,976	124,366,713	37,862,431	427.98GB

TABLE III
STATISTICAL SUMMARY OF THE QUIC-BASED DATASET

Proxy Protocol	# of Webs	# of Flow Pairs	# of Original Packets	# of Proxied Packets	File Size
SS	60	96,531	132,007,938	136,283,714	204.46GB
VMess	60	180,710	146,834,741	63,559,531	300.94GB
Trojan	60	164,223	129,782,607	124,398,554	275.33GB
VLESS	60	217,397	168,097,082	73,980,054	345.55GB

A. Dataset Collection Platform

As shown in Figure 5, we set up three hosts to build the dataset collection platform: the user, who accesses target websites; the proxy client; and the proxy server. All of these hosts are located within the same local area network. The user accesses the Internet through an HTTP/SOCKS proxy service provided by the proxy client, generating original traffic. The proxy client packages the traffic received from the user and forwards it to the proxy server using a proxy protocol, thereby creating proxied traffic. By controlling user access to network websites, we can simultaneously capture both original traffic and the corresponding proxied traffic on the proxy client. The dataset collection were conducted on a Windows 10 operating system with the Chrome browser. Since browsers disable QUIC when a proxy is configured, we must set the Proxy Client to operate in transparent proxy mode when collecting QUIC-based original and proxied traffic, ensuring that the browser functions normally.

B. Dataset Introduction

Based on the dataset collection platform, we collected a TCP-based dataset and a QUIC-based dataset, whose original traffic consists of TCP and QUIC flows, respectively. Both datasets comprise original traffic and the corresponding paired proxied traffic. Table II and Table III provide statistical

summary of the two dataset, including the number of websites, the number of flow pairs, the number of original packets, the number of proxied packets, and the file size. The proxy protocols include Shadowsocks [1], VMess [2], Trojan [3], and VLESS [4]. For both the TCP-based and QUIC-based datasets, we collect original and proxied traffic pairs simultaneously by visiting 60 popular websites 750 times for each proxy protocol.

V. COMPARATIVE EXPERIMENTAL EVALUATION

In this section, we conduct several rigorous experiments to maximize the performance of EO-EPTC. We start by introducing the evaluation preliminaries. Then, we present the comparison of Seq2Seq models. Next, we compare various attention mechanisms. Finally, we provide a detailed comparison of alignment features.

A. Preliminary

1) *Evaluation Schemes*: We conduct three comparative experiments on the TCP-based dataset constructed in the previous section, including 1) comparison of Seq2Seq models, 2) comparison of attention mechanisms, and 3) comparison of alignment features. We randomly select 30 websites from the collected TCP-based dataset, and use their original and proxied traffic to form the Unlabeled Original and Proxied Flow Pair Dataset, which is used to train the Seq2Seq model. The remaining original traffic from the other 30 websites forms the Original Traffic Dataset, while the corresponding proxied traffic is to be classified. The source code and the dataset of EO-EPTC is publicly available at <https://github.com/ygchen1/eo-epc/>.

To demonstrate the adaptability of EO-EPTC across various encrypted traffic classification models, we select the state-of-the-art sequence-based encrypted traffic classification methods FS-Net [33] and ETC-PS [35], along with traditional machine learning models Random Forest [40] and XGBoost [41], as well as the Transformer [42] model.

- FS-Net [33] employs an end-to-end encoder-decoder structure to extract features from the packet length sequence of encrypted traffic. It then performs traffic classification on a deep neural network.
- ETC-PS [35] constructs path signature features from the packet length sequences of encrypted traffic. Following this, it utilizes traditional machine learning methods to classify traffic based on the extracted features.
- Random Forest [40] uses an ensemble of decision trees to capture complex relationships between features. It aggregates the predictions from decision trees to improve overall accuracy and reduce the risk of overfitting.
- XGBoost [41] optimizes feature classification performance using the gradient-boosted tree method, which enhances the model's generalization ability.
- Transformer [42] is a sequence model designed to capture global dependencies in data through its self-attention mechanism. We leverage its encoder structure to classify the sequence features of the traffic.

We set the length of input sequence features to 32, truncating sequences longer than 32 and padding shorter ones. A

sequence length of 32 offers stable and strong classification performance across many methods [33], [35], [37], minimizing computational overhead while preserving key features for traffic classification.

The first experiment focuses on the comparison of Seq2Seq models in the sequence feature transformation modeling phase. We base our comparison on the classification performance achieved by the aforementioned encrypted traffic classification methods. The Seq2Seq models compared include LSTM [43], LSTM-Att, GRU [44], GRU-Att, and Transformer [42].

- LSTM [43] is a long short-term memory network that captures long-sequence dependencies through gating mechanisms
- LSTM-Att incorporates the attention mechanism to LSTM, which enhances its ability to focus on the most relevant parts of the input sequence.
- GRU [44] is a streamlined variant of LSTM, designed to reduce architectural complexity while preserving its ability to handle long-sequence dependencies effectively.
- GRU-Att incorporates the attention mechanism to GRU, which improves its ability to selectively focus on important input features.
- Transformer [42] is a sequence model that uses a self-attention mechanism to capture global dependencies, enabling parallel processing and efficient handling of long sequences.

The second experiment is dedicated to comparing different attention mechanisms for the Seq2Seq model. To maximize the sequence modeling capability of the Transformer, we conduct a comparative experiment on four representative attention mechanisms: Additive Attention [45], Dot-Product Attention [45], Scaled Dot-Product Attention [42], and Local Attention [46].

The third experiment aims to compare different combinations of alignment features used in the Sequence Feature Alignment phase. The optional alignment features include Directional Feature Encoding (D), Sequence Feature Reassembly (R), and Sequence Feature Segmentation (S).

- The Directional Feature Encoding (D) employs positive and negative signs to mark the packet direction in the sequence features, explicitly indicating the transmission direction information.
- The Sequence Feature Reassembly (R) is to reassemble packet-level sequence features into datagram-level sequence features affected by MTU limitations, thereby eliminating distribution differences caused by varying MTU sizes.
- The Sequence Feature Segmentation (S), which requires Sequence Feature Reassembly (R) as a prerequisite, is designed to segment the over-reassembled sequence features based on a statistical payload length threshold.

By combining these alignment steps, we can form six alignment feature combinations: (N), (D), (R), (DR), (RS), and (DRS), where (N) represents the baseline without any alignment features.

2) *Cross-Validation*: We use 5-fold cross-validation to ensure consistent and reliable dataset partitioning in all

comparative experiments. We average the results across all folds to provide a robust and comprehensive evaluation of model performance. We conduct the experiments on a general computing server with 2*Intel® Xeon® Gold 6330 CPU @ 2.00GHz, DDR4 2933MHz 256GB, and 2*NVIDIA RTX 3090 GPUs.

3) *Evaluation Criteria*: In our evaluation, we employ $F1_Macro$ and Average Normalized Edit Distance (ANED) as metrics to evaluate EO-EPTC.

- **$F1_Macro$** : We define the $precision_i$ of each Website Web_i as the ratio of correctly classified encrypted flows belonging to Web_i to the total number of encrypted flows classified as Web_i . The $recall_i$ of each Website Web_i is the ratio of correctly classified encrypted flows belonging to Web_i to the total number of encrypted flows that belong to Web_i . We calculate the $F1_score_i$ of Web_i as the harmonic mean of its $precision_i$ and $recall_i$.

$$F1_Score_i = 2 \times \frac{Precision_i \times Recall_i}{Precision_i + Recall_i} \quad (6)$$

The $F1_Macro$ is the macro average of all $F1_score_i$.

$$F1_Macro = \frac{1}{N} \sum_{i=1}^N F1_Score_i \quad (7)$$

- **ANED**: We employ Edit Distance (ED), a metric that quantifies the similarity between two sequences by calculating the minimum number of operations (insertions, deletions, substitutions) required to transform one sequence into the other. Given the sequence S_1 and S_2 , the edit distance between the first i -th element of the sequence S_1 and the first j -th characters of the sequence S_2 is

$$D(i, j) = \min \begin{cases} D(i-1, j) + 1 \\ D(i, j-1) + 1 \\ D(i-1, j-1) + 1, S_1[i] \neq S_2[j] \\ D(i-1, j-1), S_1[i] = S_2[j] \end{cases} \quad (8)$$

Thus, the edit distance between S_1 and S_2 is

$$ED(S_1, S_2) = D(|S_1|, |S_2|) \quad (9)$$

Since the length of the sequence features varies, we propose the Normalized Edit Distance (NED) to measure the degree of difference between sequence feature pairs. The NED between two sequences S_1 and S_2 is computed as:

$$NED(S_1, S_2) = \frac{ED(S_1, S_2)}{\max(\text{len}(S_1), \text{len}(S_2))} \quad (10)$$

The Average Normalized Edit Distance (ANED) is the arithmetic mean of NED across all sequence feature pairs. The $ANED$ across M sequence feature pairs is given by:

$$ANED = \frac{1}{M} \sum_{i=1}^M NED(S_1^i, S_2^i) \quad (11)$$

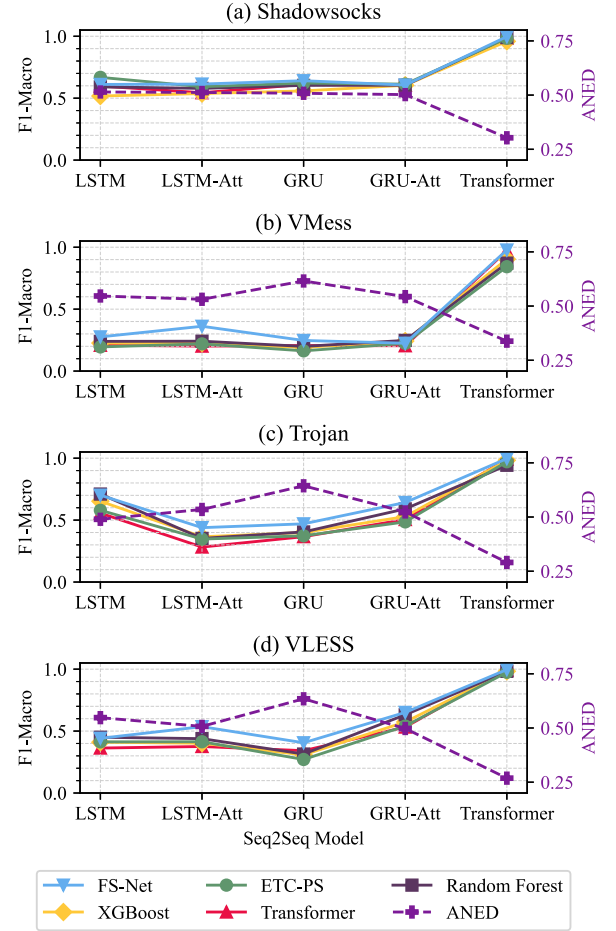


Fig. 6. Comparison of Seq2Seq model performance.

B. Comparison of Seq2Seq Models

Figure 6 illustrates the $F1_Macro$ of classifiers using EO-EPTC with various Seq2Seq models, including LSTM, LSTM-Att, GRU, GRU-Att, and Transformer, across different proxy protocols: Shadowsocks, VMess, Trojan, and VLESS. The purple dashed line in Figure 6 represents the ANED of EO-EPTC, which quantifies the similarity between the sequence features generated by EO-EPTC and those extracted from real proxy traffic. Each subfigure corresponds to a proxy protocol, with the X-axis representing the Seq2Seq method, and different line styles are used to distinguish the results of the classifiers across various Seq2Seq models.

Across all proxy protocol scenarios, the Transformer demonstrates consistent outperformance, exhibiting significantly higher $F1_Macro$ and lower ANED values compared to the other Seq2Seq models. A lower ANED indicates that the Transformer is more proficient at predicting the sequence feature of the corresponding proxy traffic. For LSTM and GRU, incorporating attention mechanisms (LSTM-Att and GRU-Att) generally improves both $F1_Macro$ scores and ANED values across most proxy protocol scenarios, with minimal variation in $F1_Macro$ scores across different classification methods. However, these performance gains fall short when compared to the Transformer. Based on the above analysis, utilizing the

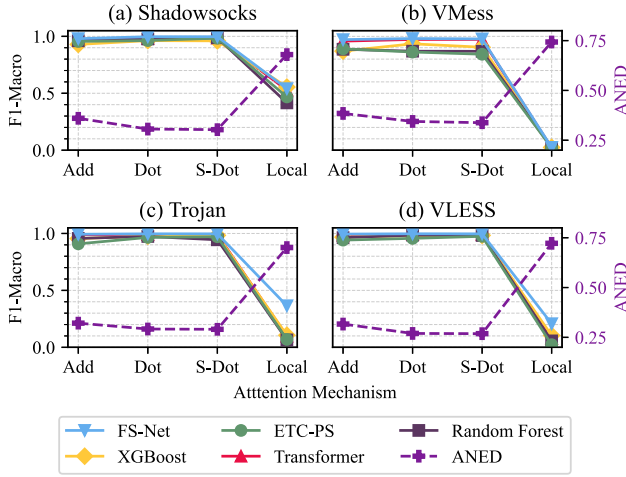


Fig. 7. Comparison of attention mechanism performance.

Transformer as the Seq2Seq model proves effective in achieving notable performance in sequence feature transformation.

C. Comparison of Attention Mechanisms

Considering the outstanding performance of the Transformer demonstrated in Section V-B, we further investigate the impact of various Transformer attention mechanisms on the performance of EO-EPTC. Figure 7 shows the F1-Macro of classifiers using the Transformer as EO-EPTC's Seq2Seq model with various attention mechanisms, including Additive [45], Dot-Product [45], Scaled Dot-Product [42], and Local [46] attention mechanisms. Similar to Section V-B, the purple dashed line in Figure 7 represents the ANED of EO-EPTC. Each subfigure represents a proxy protocol, where the X-axis depicts the attention mechanism. Different line styles are employed to differentiate the classifier results for various attention mechanisms.

In all proxy protocol scenarios, the Transformer with Scaled Dot-Product Attention consistently shows the lowest ANED values and allows classifiers to achieve the highest F1-Macro scores. Dot-Product Attention and Additive Attention perform slightly worse than Scaled Dot-Product Attention, while Local Attention displayed relatively weak performance. Compared to other attention mechanisms, the Scaled Dot-Product Attention offers enhanced global context awareness by efficiently scaling the dot products, which reduces the impact of large value ranges and ensures a more stable and informative attention distribution. In contrast, the Local Attention with a limited context window struggles to model cross-timestep dependencies, especially in complex transmission proxy protocols like VMess.

In summary, the Scaled Dot-Product Attention proves to be the most effective mechanism within the EO-EPTC, excelling in both F1-Macro scores and ANED quality. This reinforces the Transformer's capability to capture complex sequence features effectively.

D. Comparison of Alignment Features

Figure 8 demonstrates the performance of different alignment feature combinations across four proxy protocols, with

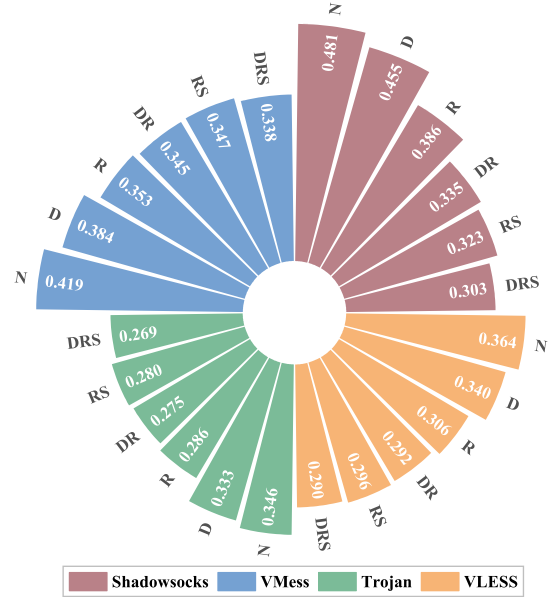


Fig. 8. Evaluation of alignment feature combinations.

ANED as the only evaluation metric. The alignment features compared include Directional Feature Encoding (D), Sequence Feature Reassembly (R), Sequence Feature Segmentation (S), and their combinations, such as DR, RS, and DRS. Notably, Sequence Feature Segmentation (S) requires Sequence Feature Reassembly (R) as a prerequisite. We use (N) to represent the baseline without any alignment features.

The (DRS) combination of all three alignment features consistently achieves the lowest ANED values across all protocols. The (RS) also performs well, with ANED values slightly higher than DRS but still lower than other combinations. This suggests that even without Directional Feature Encoding, the combination of Sequence Feature Reassembly and Sequence Feature Segmentation substantially enhances sequence alignment. The (DR) configuration shows slightly weaker performance compared to RS but still outperforms using (D) or (R) alone. Both (D) and (R), when applied individually, provide slight improvements over the baseline (N). This highlights that only the combination of Directional Feature Encoding and Sequence Feature Reassembly can achieve a significant improvement in sequence alignment.

The results demonstrate that the combination of all three alignment features (DRS) can greatly improve the EO-EPTC framework's effectiveness in sequence feature prediction, which in turn influences the training process for proxy traffic classification.

VI. PERFORMANCE EVALUATION FOR EO-EPTC

In this section, we use the optimal configuration of EO-EPTC demonstrated in Section V to evaluate the robustness and effectiveness of EO-EPTC. We begin by introducing the evaluation preliminaries, and then present a series of experiments designed to examine EO-EPTC from multiple perspectives.

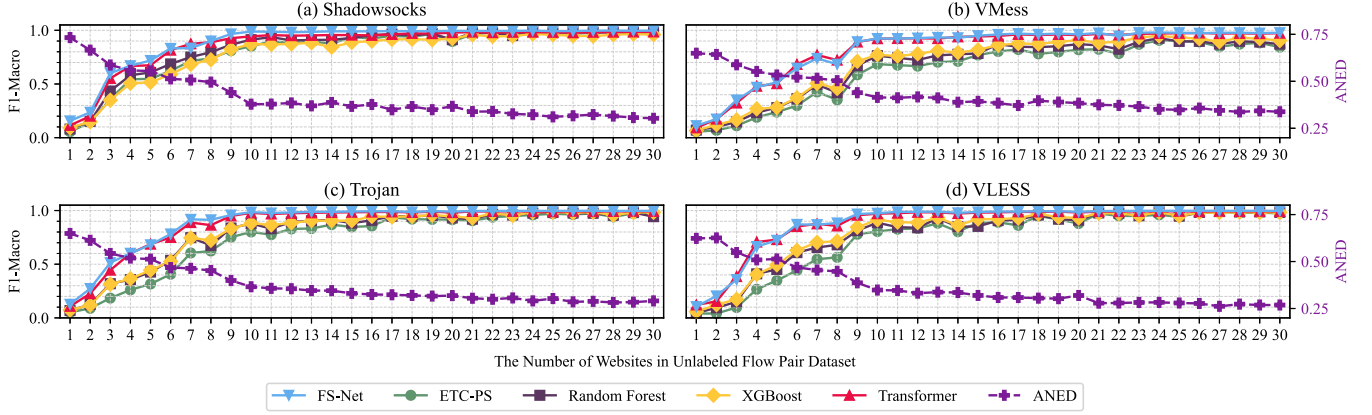


Fig. 9. Comparison of the impact of dataset scale on the unlabeled flow pair dataset.

A. Preliminary

We conduct three evaluation experiments to assess the effectiveness of the EO-EPTC configurations demonstrated in the previous section, including 1) the impact of dataset scale on EO-EPTC, 2) performance improvement with EO-EPTC, 3) the compatibility to QUIC-based proxied traffic, 4) the effectiveness of out-of-distribution traffic detection, 5) the impact of different operating systems and browsers to EO-EPTC, and 6) comparison with representative baseline approaches.

The first experiment focuses on the impact of dataset scale on EO-EPTC. In previous experiments, the EO-EPTC uses the TCP-based Flow Pair Dataset from 30 websites to train the Seq2Seq model. In this experiment, we evaluate the F1-Macro and ANED of classifiers using unlabeled flow pair datasets constructed with 1 to 30 websites.

In the second experiment, we evaluate the performance improvement achieved by the EO-EPTC. We use TCP-based dataset to compare the encrypted traffic classifiers under three training scenarios: original traffic, real-world proxied traffic, and original traffic combined with EO-EPTC. Subsequently, we assess the classifiers' performance using real-world proxied traffic to determine the enhancement brought about by EO-EPTC.

In the third experiment, we evaluate the compatibility of EO-EPTC to QUIC-based proxied traffic. Similar to TCP-based traffic, we first train a seq2seq model and transform original sequence features derived from QUIC traffic into simulated proxied sequence features, then train the classifier using these simulated features. We evaluate the classifiers' performance directly on the proxied traffic carrying QUIC.

In the fourth experiment, we evaluate the effectiveness of out-of-distribution (OOD) traffic detection. We divide the labeled TCP-based dataset into three mutually exclusive subsets: known classes, simulated OOD classes, and real OOD classes. We first train a traffic classifier using the simulated proxied sequence features generated from the original traffic of the known classes. Then, we train an OOD detector, implemented as a random forest classifier, using the probability vectors produced by the traffic classifier for the original traffic of both the known classes and the simulated OOD classes. The

overall detection and classification performance is evaluated on the proxied traffic samples from the known classes and the real OOD classes.

In the fifth experiment, we evaluate the impact of different operating systems and browsers on the performance of EO-EPTC. We construct TCP-based datasets on four platform combinations by considering two operating systems, Windows and Ubuntu, and two browsers, Chrome and Firefox. For each platform, we collect the corresponding traffic datasets and assess the classifiers under the same experimental settings as in the original submission.

In the sixth experiment, we compare EO-EPTC with three representative baseline VPN classification approaches. We select Flowpic [20], [21], Flowformers [22] and Minipic [23] as baseline methods. We use the original traffic from TCP-based dataset as the training set for the baseline models and evaluate their performance using the proxied traffic. The results are compared with the performance of the EO-EPTC-enhanced FS-Net method.

B. Impact of Dataset Scale on EO-EPTC

Figure 7 presents the F1-Macro and ANED of classifiers as the number of websites in the unlabeled flow pair dataset varies from 1 to 30. Similar to the previous figures, the purple dashed line in Figure 7 represents the ANED of EO-EPTC. Each subfigure corresponds to a proxy protocol, with the X-axis representing the number of websites in the unlabeled flow pair dataset. We use different line styles to distinguish the results of the state-of-the-art classifiers.

The F1-Macro exhibits a clear upward trend as the number of websites in the dataset increases. The most significant improvement of F1-Macro is observed as the number of websites increases from 1 to 10. When the number of websites exceeds 26, the F1-Macro tends to stabilize at a high level. As the number of websites increases, the Seq2Seq model gradually shifts from learning the transformation rules of sequence features for specific websites to understanding the implicit semantics of proxy protocols. This further enables the transformation of sequence features for the unseen websites.

In terms of ANED, we observe a consistent downward trend, which also implies that the capacity to predict sequence

TABLE IV

CLASSIFICATION PERFORMANCE IMPROVEMENT WITH EO-EPTC

Classifier	EO-EPTC	Train Data	Test Data	SS	VMess	Trojan	VLESS
FS-Net	–	Original	Proxied	4.02%	4.93%	4.82%	4.08%
FS-Net	–	Proxied	Proxied	99.93%	99.43%	99.97%	99.96%
FS-Net	✓	Original	Proxied	99.70%	97.75%	99.67%	99.67%
ETC-PS	–	Original	Proxied	1.48%	2.22%	1.66%	2.45%
ETC-PS	–	Proxied	Proxied	99.87%	99.35%	99.58%	99.82%
ETC-PS	✓	Original	Proxied	98.40%	84.29%	97.14%	97.48%
RF	–	Original	Proxied	2.96%	1.54%	1.46%	1.11%
RF	–	Proxied	Proxied	99.88%	99.32%	99.92%	99.90%
RF	✓	Original	Proxied	98.84%	87.11%	94.55%	98.39%
XGBoost	–	Original	Proxied	4.25%	3.00%	3.99%	2.87%
XGBoost	–	Proxied	Proxied	99.15%	98.81%	99.53%	99.47%
XGBoost	✓	Original	Proxied	96.04%	90.43%	98.32%	98.31%
Transformer	–	Original	Proxied	4.45%	4.23%	4.07%	4.72%
Transformer	–	Proxied	Proxied	99.42%	98.89%	99.57%	99.56%
Transformer	✓	Original	Proxied	98.51%	97.31%	99.21%	99.20%

features improves with the expansion of the unlabeled flow pair dataset. Notably, both Trojan and VLESS exhibit more pronounced declines in ANED, highlighting that protocols with more complex transmission patterns derive significant benefits from the enlargement of the dataset.

In summary, the expansion of the unlabeled flow pair dataset enhances the Seq2Seq model's ability to learn the implicit semantics of proxy protocols. As the dataset becomes more diverse, the model's performance stabilizes, indicating that it learns a generalizable transformation function rather than overfitting to specific website patterns. This enables the model to handle out-of-distribution scenarios and establish a universal mapping for unseen websites. As a result, the model accurately converts original traffic features, allowing classifiers trained on this simulated data to effectively classify real-world encrypted proxy traffic.

C. Performance Improvement With EO-EPTC

Table IV presents the detained classification performance of five state-of-the-art classifiers across three training scenarios: 1) original → proxied, 2) proxied → proxied, and 3) original + EO-EPTC → original.

The classifiers trained solely on original traffic exhibit poor classification performance when predicted on the proxied traffic, with F1-Macro scores ranging from 1.46% to 4.93%. This is primarily attributed to the distribution discrepancy between original and proxied traffic, as well as the requirement for machine learning models to ensure consistency in the distribution of input data. When EO-EPTC is applied to the training dataset, classifiers trained on original traffic achieve F1-Macro scores that closely align with those of models trained directly on proxied traffic. FS-Net's performance on Shadowsocks improved dramatically from 4.02% to 99.70% when trained with original traffic using EO-EPTC, coming within 0.23% of the 99.93% achieved by training directly on proxied traffic. Similarly, XGBoost's performance on Trojan traffic rose significantly from 3.99% to 98.32%, closely matching the 99.53% accuracy obtained through direct training on proxied traffic. The classification performance improvement

TABLE V

CLASSIFICATION PERFORMANCE TO QUIC-BASED DATASET

Classifier	EO-EPTC	Train Data	Test Data	SS	VMess	Trojan	VLESS
FS-Net	–	Original	Proxied	12.56%	10.50%	12.51%	9.37%
FS-Net	–	Proxied	Proxied	99.32%	98.77%	98.49%	98.71%
FS-Net	✓	Original	Proxied	93.70%	85.79%	86.87%	85.77%

TABLE VI

CLASSIFICATION PERFORMANCE TO OOD TRAFFIC

Division Scheme	SS	VMess	Trojan	VLESS
ABCD + E + F	89.30%/71.55%	89.19%/78.75%	91.30%/85.31%	93.42%/86.22%
ABCD + F + E	90.00%/70.77%	93.12%/81.78%	93.33%/87.41%	93.04%/85.91%
CDEF + A + B	82.31%/88.74%	82.01%/84.95%	90.33%/93.01%	88.64%/92.39%
CDEF + B + A	90.48%/90.15%	89.13%/90.21%	93.53%/94.22%	91.62%/93.33%

achieved by EO-EPTC benefits not only state-of-the-art methods and Transformers but also traditional machine learning models, such as Random Forest and XGBoost.

These results underscore EO-EPTC's ability to generate simulated sequence features of proxied traffic that closely resemble those of real proxied traffic. This capability allows existing classifiers to perform effective encrypted proxy traffic classification using only original traffic for training.

D. Compatibility to QUIC-Based Proxied Traffic

Table V presents the classification performance when applying EO-EPTC to QUIC-based traffic. The F1-Macro score ranges from 85.77% to 93.70% when training the FS-Net on the original traffic and evaluating it on Proxied traffic. While this result is slightly lower than when training directly on Proxied traffic, it remains significantly higher than the performance obtained by training on original traffic without the use of EO-EPTC. This reduction is due to the fact that QUIC traffic, which is based on the UDP protocol, does not provide full flow control information, making it difficult to perform accurate flow reassembly and mitigate traffic noise caused by network fluctuations. This issue may be partially alleviated by increasing the volume of data collected for training.

E. Detection of OOD Traffic

Table VI presents the classification performance in scenarios involving OOD (Out-Of-Distribution) traffic. We evenly divided the 30 website categories into six subsets (A, B, C, D, E, F), generating different combinations of known classes, simulated OOD classes, and real OOD classes for evaluation. We assessed the classification performance under four different combinations. In each result cell, the first value represents the overall F1-Macro score for the 21 classes (20 categories + 1 OOD category), while the second value represents the F1-score for the OOD category.

We used FS-Net as the traffic classifier and random forest as the OOD traffic detection classifier. It can be observed that, in the OOD traffic scenario, the FS-Net enhanced with EO-EPTC still achieves a strong overall F1-Macro score and OOD category F1-score. This indicates that EO-EPTC

TABLE VII
SUMMARY OF THE DATASET FROM DIFFERENT PLATFORMS

Proxy Protocol	# of Webs	# of Flow Pairs	# of Original Packets	# of Proxied Packets	File Size
Platform: Windows 10 + Firefox					
SS	60	1,252,094	267,309,903	278,664,488	914.68GB
VMess	60	1,249,581	273,982,508	73,586,974	936.41GB
Trojan	60	1,267,523	272,202,093	71,811,519	938.34GB
VLESS	60	1,253,247	274,061,527	71,870,963	934.61GB
Platform: Ubuntu 22.04 + Chrome					
SS	60	1,078,701	208,986,094	223,200,491	817.69GB
VMess	60	946,598	212,197,293	58,420,877	820.57GB
Trojan	60	970,364	220,158,295	69,691,214	765.28GB
VLESS	60	968,887	213,346,035	58,590,903	824.22GB
Platform: Ubuntu 22.04 + Firefox					
SS	60	1,208,560	271,116,772	278,674,528	997.49GB
VMess	60	1,202,070	289,546,764	80,960,890	1002.99GB
Trojan	60	1,225,216	291,977,454	80,359,211	998.74GB
VLESS	60	1,217,050	284,443,237	78,970,794	971.84GB

TABLE VIII
CROSS-PLATFORM PERFORMANCE OF EO-EPTC

Operating System	Browser	SS	VMess	Trojan	VLESS
Windows 10	Chrome	99.70%	97.75%	99.67%	99.67%
Windows 10	Firefox	98.00%	94.97%	99.09%	99.13%
Ubuntu 22.04	Chrome	96.27%	92.65%	98.27%	98.08%
Ubuntu 22.04	Firefox	97.45%	90.98%	98.12%	98.22%

effectively simulates the characteristics of real proxy traffic, thereby supporting the identification of OOD traffic.

F. Cross-Platform Performance of EO-EPTC

We evaluate the classification performance of EO-EPTC across four platform combinations: Windows 10 + Chrome, Windows 10 + Firefox, Ubuntu 22.04 + Chrome, and Ubuntu 22.04 + Firefox. The data collection procedures and proxy protocols remain consistent with the original setup to ensure comparability. Table VII provides a summary of the extra dataset.

Table VIII presents the cross-platform performance of EO-EPTC. EO-EPTC achieves consistently high accuracy across all platforms, with performance exceeding 90% in every case. Minor fluctuations are observed between Windows and Ubuntu, primarily for the VMess protocol, which may be attributed to differences in TCP stack implementations that lead to additional instances of merging and misalignment in sequence features. In contrast, the performance gap between Chrome and Firefox is relatively negligible.

Overall, these findings confirm that feature distinction is primarily determined by the inherent characteristics of proxy protocols, while the influence of the operating system or browser remains minimal. These observations highlight the robustness and effectiveness of EO-EPTC when applied to real-world encrypted proxy traffic classification.

G. Comparison With Baseline Approaches

We select three baseline approaches for comparison: Flowpic, which uses statistical analysis of packet timing and

TABLE IX
COMPARISON WITH BASELINE APPROACHES

Approach	SS	VMess	Trojan	VLESS
FlowPic	8.39%	8.31%	8.46%	8.74%
Flowformers	7.61%	6.39%	7.69%	8.51%
mini-FlowPic	17.99%	26.94%	15.69%	25.22%
FS-Net+EO-EPTC	99.70%	97.75%	99.67%	99.67%

payload information to generate an image-like representation, and then applies CNN for classification. Flowpic claims to be capable of classifying VPN traffic when trained on non-VPN traffic, which shares some similarities with our scenario. Flowformers takes into account the traffic direction and different time-length statistical methods to construct multi-channel feature representations. mini-FlowPic is an improvement on Flowpic, increasing the granularity of statistics to achieve more efficient analysis and classification, thereby reducing the dependence on dataset size.

Table IX presents the comparison results with the baseline approaches. Among the three baseline methods, mini-FlowPic performs relatively better. However, compared to the FS-Net enhanced with EO-EPTC, the baseline methods show relatively lower classification performance on proxied traffic based on original traffic. These methods struggle to overcome the challenges posed by the data distribution differences.

VII. CONCLUSION AND FUTURE WORK

In this work, we propose EO-EPTC, an end-to-end original traffic-based encrypted proxy traffic classification framework that bridges the distribution gap via sequence feature alignment and a Seq2Seq model, enabling models trained only on original traffic to classify proxied traffic. Although the proposed method demonstrates promising results, the current model assumes a one-to-one mapping between original flows and proxy flows. However, some proxy protocols, such as Hysteria2, employ multiplexing over QUIC-based HTTP/3, where multiple application flows are encapsulated within a single proxy flow. This many-to-one mapping increases the instability of length sequence features, making accurate identification more challenging. In future work, we plan to expand our dataset with multiplexed traffic and investigate temporal slicing techniques to isolate sub-flows, thereby improving recognition performance in such scenarios.

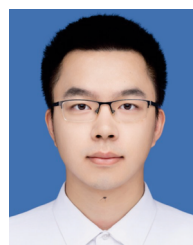
REFERENCES

- [1] (2024). *Shadowsocks—A Fast Tunnel Proxy That Helps You Bypass Firewalls*. [Online]. Available: <https://shadowsocks.org/>
- [2] (2024). *VMess*. [Online]. Available: <https://www.v2ray.com/chapter02/protocols/vmess.html>
- [3] (2024). *Trojan Documentation*. [Online]. Available: <https://trojan-gfw.github.io/trojan/>
- [4] (2024). *VLESS*. [Online]. Available: <https://www.v2fly.org/config/protocols/vless.html>
- [5] A. H. Lashkari, G. D. Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Tor traffic using time based features," in *Proc. 3rd Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, Feb. 2017, pp. 253–262.
- [6] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of encrypted and VPN traffic using time-related," in *Proc. 2nd Int. Conf. Inf. Syst. Security Privacy (ICISSP)*, 2016, pp. 407–414.

- [7] P. Lin, K. Ye, Y. Hu, Y. Lin, and C.-Z. Xu, "A novel multimodal deep learning framework for encrypted traffic classification," *IEEE/ACM Trans. Netw.*, vol. 31, no. 3, pp. 1369–1384, Jun. 2023.
- [8] X. Yun, Y. Wang, Y. Zhang, C. Zhao, and Z. Zhao, "Encrypted TLS traffic classification on cloud platforms," *IEEE/ACM Trans. Netw.*, vol. 31, no. 1, pp. 164–177, Feb. 2023.
- [9] Y. Wang, H. He, Y. Lai, and A. X. Liu, "A two-phase approach to fast and accurate classification of encrypted traffic," *IEEE/ACM Trans. Netw.*, vol. 31, no. 3, pp. 1071–1086, Jun. 2023.
- [10] X. Ma et al., "Context-aware website fingerprinting over encrypted proxies," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, May 2021, pp. 1–10.
- [11] H. Zhao, S. Zhang, Z. Qiao, X. Huang, and X. Zhang, "On the performance of deep learning methods for identifying abnormal encrypted proxy traffic," in *Proc. IEEE Int. Conf. Trust, Secur. Privacy Comput. Commun. (TrustCom)*, Dec. 2022, pp. 1416–1423.
- [12] X. Ma et al., "Website fingerprinting on encrypted proxies: A flow-context-aware approach and countermeasures," *IEEE/ACM Trans. Netw.*, vol. 32, no. 3, pp. 1904–1919, Jun. 2024.
- [13] D. Xue, M. Kallitsis, A. Houmansadr, and R. Ensafi, "Fingerprinting obfuscated proxy traffic with encapsulated TLS handshakes," in *Proc. 33rd USENIX Secur. Symp. (USENIX Secur.)*, 2024, pp. 2689–2706.
- [14] A. Montieri, D. Ciunzo, G. Bovenzi, V. Persico, and A. Pescapé, "A dive into the dark Web: Hierarchical traffic classification of anonymity tools," *IEEE Trans. Netw. Sci. Eng.*, vol. 7, no. 3, pp. 1043–1054, Jul. 2020.
- [15] Y. Li et al., "From traffic classes to content: A hierarchical approach for encrypted traffic classification," *Comput. Netw.*, vol. 212, Jul. 2022, Art. no. 109017.
- [16] Z. Wu, Y.-N. Dong, X. Qiu, and J. Jin, "Online multimedia traffic classification from the QoS perspective using deep learning," *Comput. Netw.*, vol. 204, Feb. 2022, Art. no. 108716.
- [17] Z. Song et al., "I² RNN: An incremental and interpretable recurrent neural network for encrypted traffic classification," *IEEE Trans. Dependable Secure Comput.*, early access, Feb. 28, 2024, doi: [10.1109/TDSC.2023.3245411](https://doi.org/10.1109/TDSC.2023.3245411).
- [18] F. Cerasuolo et al., "MEMENTO: A novel approach for class incremental learning of encrypted traffic," *Comput. Netw.*, vol. 245, May 2024, Art. no. 110374.
- [19] Y. Zhong, Z. Wang, X. Shi, J. Yang, and K. Li, "RFG-HELAD: A robust fine-grained network traffic anomaly detection model based on heterogeneous ensemble learning," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 5895–5910, 2024.
- [20] T. Shapira and Y. Shavitt, "FlowPic: Encrypted Internet traffic classification is as easy as image recognition," in *Proc. IEEE INFOCOM-IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, Apr. 2019, pp. 680–687.
- [21] T. Shapira and Y. Shavitt, "FlowPic: A generic representation for encrypted traffic classification and applications identification," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 2, pp. 1218–1232, Jun. 2021.
- [22] R. Babaria, S. C. Madanapalli, H. Kumar, and V. Sivaraman, "FlowFormers: Transformer-based models for real-time network flow classification," in *Proc. 17th Int. Conf. Mobility, Sens. Netw. (MSN)*, Dec. 2021, pp. 231–238.
- [23] E. Horowicz, T. Shapira, and Y. Shavitt, "A few shots traffic classification with mini-FlowPic augmentations," in *Proc. 22nd ACM Internet Meas. Conf.*, 2022, pp. 647–654.
- [24] G. Aceto, D. Ciunzo, A. Montieri, A. Nascita, and A. Pescapé, "Encrypted multitask traffic classification via multimodal deep learning," in *Proc. ICC-IEEE Int. Conf. Commun.*, Jun. 2021, pp. 1–6.
- [25] X. Xiao, W. Xiao, R. Li, X. Luo, H. Zheng, and S. Xia, "EBSNN: Extended byte segment neural network for network traffic classification," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 5, pp. 3521–3538, Sep. 2022.
- [26] G. Xie, Q. Li, and Y. Jiang, "Self-attentive deep learning method for online traffic classification and its interpretability," *Comput. Netw.*, vol. 196, Sep. 2021, Art. no. 108267.
- [27] I. Guarino, G. Aceto, D. Ciunzo, A. Montieri, V. Persico, and A. Pescapé, "Contextual counters and multimodal deep learning for activity-level traffic classification of mobile communication apps during COVID-19 pandemic," *Comput. Netw.*, vol. 219, Dec. 2022, Art. no. 109452.
- [28] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf.*, Apr. 2022, pp. 633–642.
- [29] Y. Xiao et al., "Distributed traffic synthesis and classification in edge networks: A federated self-supervised learning approach," *IEEE Trans. Mobile Comput.*, vol. 23, no. 2, pp. 1815–1829, Feb. 2024.
- [30] J. Yu, Y. Choi, K. Koo, and D. Moon, "A novel approach for application classification with encrypted traffic using BERT and packet headers," *Comput. Netw.*, vol. 254, Dec. 2024, Art. no. 110747.
- [31] Y. Liu, X. Wang, B. Qu, and F. Zhao, "ATVITSC: A novel encrypted traffic classification method based on deep learning," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 9374–9389, 2024.
- [32] X. Zhang et al., "Enhanced few-shot malware traffic classification via integrating knowledge transfer with neural architecture search," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 5245–5256, 2024.
- [33] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-Net: A flow sequence network for encrypted traffic classification," in *Proc. IEEE INFOCOM-IEEE Conf. Comput. Commun.*, Apr. 2019, pp. 1171–1179.
- [34] M. Jiang et al., "Accurate mobile-app fingerprinting using flow-level relationship with graph neural networks," *Comput. Netw.*, vol. 217, Nov. 2022, Art. no. 109309.
- [35] S.-J. Xu, G.-G. Geng, X.-B. Jin, D.-J. Liu, and J. Weng, "Seeing traffic paths: Encrypted traffic classification with path signature features," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 2166–2181, 2022.
- [36] J. Piet, D. Nwoji, and V. Paxson, "GGFAST: Automating generation of flexible network traffic classifiers," in *Proc. ACM SIGCOMM Conf.*, 2023, pp. 850–866.
- [37] J. Qu et al., "An input-agnostic hierarchical deep learning framework for traffic fingerprinting," in *Proc. 32nd USENIX Secur. Symp. (USENIX Secur.)*, 2023, pp. 589–606.
- [38] Q. Zhou, L. Wang, H. Zhu, T. Lu, and V. S. Sheng, "WF-transformer: Learning temporal features for accurate anonymous traffic identification by using transformer networks," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 30–43, 2024.
- [39] (2024). V2ray. [Online]. Available: <https://www.v2ray.com/>
- [40] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.
- [41] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2016, pp. 785–794.
- [42] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 5998–6008.
- [43] S. Hochreiter, "Long short-term memory," in *Neural Computation*. Cambridge, MA, USA: MIT Press, 1997.
- [44] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," 2014, *arXiv:1412.3555*.
- [45] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *Proc. 3rd Int. Conf. Learn. Represent.*, 2014, pp. 1–11.
- [46] T. Luong, H. Pham, and C. D. Manning, "Effective approaches to attention-based neural machine translation," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2015, pp. 1412–1421.



Yige Chen received the Ph.D. degree in computer software and theory from the Institute of Information Engineering, Chinese Academy of Sciences (CAS), China, in 2022. He is currently a Lecturer with the College of Computer Science and Artificial Intelligence, Wenzhou University, China. His research interests include encrypted traffic fingerprints, network security, and machine learning.



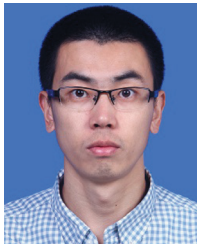
Huajie Jia received the bachelor's degree in computer science and artificial intelligence from Wenzhou University, China, in 2023, where he is currently pursuing the master's degree with the College of Computer Science and Artificial Intelligence. His research interests include encrypted traffic fingerprints, network security, and machine learning.



Zhenzhou Tang (Senior Member, IEEE) received the B.S. and M.S. degrees in communication engineering from Zhejiang University in 2001 and 2004, respectively, and the Ph.D. degree in communication engineering from Dalian University of Technology in 2015. From February 2015 to August 2015, he was a Visiting Scholar with the Department of Electrical and Computer Engineering, Worcester Polytechnic Institute, Worcester, MA, USA. From November 2015 to June 2018, he was a Post-Doctoral Research Fellow with Zhejiang University.

He is currently a Full Professor with the College of Computer Science and Artificial Intelligence, Wenzhou University, and the Director of Wenzhou Key Laboratory for Intelligent Networking. His current research interests include machine learning, intelligent systems, and wireless networks.

INFORMATION FORENSICS AND SECURITY, IEEE/ACM TRANSACTIONS ON NETWORKING, IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE TRANSACTIONS ON COGNITIVE COMMUNICATIONS AND NETWORKING, IEEE TRANSACTIONS ON NETWORK SCIENCE AND ENGINEERING, and IEEE INTERNET OF THINGS JOURNAL.



Yipeng Wang (Senior Member, IEEE) received the Ph.D. degree in computer science from the Institute of Computing Technology, Chinese Academy of Sciences (CAS), China, in 2014. He is currently a Full Professor at the College of Computer Science, Beijing University of Technology, China. He has published more than 50 research papers in refereed international journals and conferences, such as IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE/ACM TRANSACTIONS ON NETWORKING, IEEE ICNP, and IEEE

TRANSACTIONS ON NETWORK AND SERVICE MANAGEMENT. His research interests include networking, network security, and machine learning. He was a recipient of the Best Paper Award at the IEEE International Conference on Network Protocols (ICNP) for his protocol format inference technology. He serves as a Regular Reviewer for IEEE TRANSACTIONS ON



Hui Liu received the master's degree in computer science from the Institute of Information Engineering, Chinese Academy of Sciences (CAS), China, in 2020. He is currently a Researcher with Tencent, Beijing, China. His research interests include large language models, natural language generation, and machine learning.